



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

Практична робота №3
Моделі математичної економіки та прийняття рішень в
інформаційних системах

Тема: Прийняття рішень в задачах багатокритеріальної оптимізації

Варіант 6

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Поперешняк С. В.

ЗМІСТ

1 Мета	3
2 Постановка задачі (варіант 6)	4
3 Нормалізація критеріїв	5
4 Метод згортки	6
5 Метод головного критерію	7
6 Метод послідовної поступки	8
7 Аналіз результатів	9
8 Висновки	10
9 Лістинг коду	11
9.1 multi_criteria.py	11

1 МЕТА

Вивчити методи розв'язання задач багатокритеріальної оптимізації: метод згортки, метод головного критерію та метод послідовної поступки.

Усі розрахунки виконано мовою Python (бібліотека SciPy, функція `linprog` для розв'язання задач лінійного програмування) замість надбудови Excel «Пошук рішення», а звіт підготовлено засобами Typst замість MS Word.

2 ПОСТАНОВКА ЗАДАЧІ (ВАРІАНТ 6)

Три комбайни (I, II, III) виконують три види робіт (A, B, C). Задано продуктивність q_{ij} (м³/год), питому вартість c_{ij} (грн/год) та ресурс часу кожного комбайна:

Таблиця 2.1 Вихідні дані варіанта 6: продуктивність, вартість, ресурс часу

Комбайн	q_A	q_B	q_C	c_A	c_B	c_C	Ресурс, год
I	30	20	40	3	6	4	450
II	40	30	20	4	2	3	320
III	20	40	30	5	3	2	300

Позначимо через x_{ij} час виконання i -м комбайном j -ї роботи. Математична модель задачі:

$$f_1(x) = \sum_{i,j} q_{ij} x_{ij} \rightarrow \max;$$

$$f_2(x) = \sum_{i,j} c_{ij} x_{ij} \rightarrow \min;$$

$$\sum_j x_{ij} \leq T_i, \quad i = \text{I, II, III}; \quad x_{ij} \geq 0.$$

3 НОРМАЛІЗАЦІЯ КРИТЕРІЇВ

Для застосування методу згортки критерії потрібно звести до безрозмірного вигляду. Спочатку знайдено максимальні й мінімальні значення кожного критерію на множині припустимих альтернатив (розв'язанням відповідних однокритеріальних задач лінійного програмування):

Таблиця 3.1 Максимальні та мінімальні значення критеріїв

Критерій	Максимум	Мінімум
f_1 (обсяг робіт)	42800	0
f_2 (вартість)	5480	0

Таблиця 3.2 Розв'язок задачі максимізації f_1 ($f_1^{\max} = 42800$ м³; супутня вартість $f_2 = 3980$ грн)

Комбайн	A	B	C	Реальний	За умовою
I	0	0	450	450	≤ 450
II	320	0	0	320	≤ 320
III	0	300	0	300	≤ 300

Таблиця 3.3 Розв'язок задачі максимізації f_2 ($f_2^{\max} = 5480$ грн)

Комбайн	A	B	C	Реальний	За умовою
I	0	450	0	450	≤ 450
II	320	0	0	320	≤ 320
III	300	0	0	300	≤ 300

Оскільки f_1 максимізується, а найгірше (мінімальне) значення дорівнює 0, нормалізований критерій має вигляд:

$$f_1^*(x) = \frac{f_1^{\max} - f_1(x)}{f_1^{\max}} = \frac{42800 - f_1(x)}{42800} \rightarrow \min.$$

Оскільки f_2 мінімізується, а найгірше (максимальне) значення дорівнює 5480, нормалізований критерій:

$$f_2^*(x) = \frac{f_2(x) - f_2^{\min}}{f_2^{\max}} = f_2 \frac{x}{5480} \rightarrow \min.$$

4 МЕТОД ЗГОРТКИ

Прийнято вагові коефіцієнти значущості критеріїв $\alpha_1 = 0.5$, $\alpha_2 = 0.5$ (рівнозначні критерії), оскільки обидва критерії лінійні, застосовано лінійну адитивну згортку:

$$F(x) = \alpha_1 f_1^*(x) + \alpha_2 f_2^*(x) \rightarrow \min.$$

Розв'язання відповідної скалярної задачі лінійного програмування дає:

Таблиця 4.1 Розв'язок методом згортки ($\alpha_1=0.5$, $\alpha_2=0.5$)

Комбайн	A	B	C	Реальний	За умовою
I	0	0	450	450	≤ 450
II	0	320	0	320	≤ 320
III	0	300	0	300	≤ 300

Отримано $f_1 = 39600 \text{ м}^3$, $f_2 = 3340 \text{ грн}$, значення узагальненого критерію $F = 0.342128$.

Вплив вибору вагових коефіцієнтів. Якщо, за прикладом методичних вказівок, обрати ваги $\alpha_1 = 0.7$, $\alpha_2 = 0.3$ (значно вищий пріоритет обсягу робіт), розв'язок дає той самий вектор x , що й чиста максимізація f_1 ($x = (0, 0, 450, 320, 0, 0, 0, 300, 0)$, $f_1 = 42800$, $f_2 = 3980$). Це пояснюється тим, що обмеження задачі є повністю незалежними по кожному комбайну (кожен рядок x_i фігурує лише у своєму обмеженні), тому оптимум лінійної згортки завжди досягається у вершині багатогранника — розподілі всього ресурсу комбайна на одну роботу; за досить великої ваги α_1 ця вершина збігається з вершиною чистої максимізації f_1 . Щоб отримати змістовний компроміс (а не вироджений збіг з однокритеріальним розв'язком), у цьому звіті використано рівнозначні ваги $\alpha_1 = \alpha_2 = 0.5$.

5 МЕТОД ГОЛОВНОГО КРИТЕРІЮ

Головним критерієм обрано f_1 (обсяг робіт) як такий, для якого експертна оцінка допустимого рівня є найскладнішою. Встановлено додаткове обмеження на максимально допустиму вартість робіт $f_2(x) \leq 3000$ грн:

$$f_1(x) \rightarrow \max; \quad f_2(x) \leq 3000; \quad \sum_j x_{ij} \leq T_i; \quad x_{ij} \geq 0.$$

Таблиця 5.1 Розв'язок методом головного критерію ($f_2 \leq 3000$)

Комбайн	A	B	C	Реальний	За умовою
I	340	0	110	450	≤ 450
II	0	320	0	320	≤ 320
III	0	300	0	300	≤ 300

Отримано $f_1 = 36200$ м³, $f_2 = 3000$ грн (обмеження вартості виконується як рівність, тобто ресурс вартості вичерпано повністю).

6 МЕТОД ПОСЛІДОВНОЇ ПОСТУПКИ

Спочатку розв'язано скалярну задачу за найбільш значущим критерієм f_1 на всій множині припустимих альтернатив (розв'язок наведено в таблиці розв'язку максимізації f_1 вище): $f_1^{\max} = 42800 \text{ м}^3$.

Прийнято припустиму поступку за першим критерієм $\Delta = 6976 \text{ м}^3$ (16.3% від $f_1^{\max}$), тобто нова нижня межа: $f_1(x) \geq f_1^{\max} - \Delta = 35824$. На другому кроці розв'язано задачу мінімізації другого критерію за цього додаткового обмеження:

$$f_2(x) \rightarrow \min; \quad f_1(x) \geq 35824; \quad \sum_j x_{ij} \leq T_i; \quad x_{ij} \geq 0.$$

Таблиця 6.1 Розв'язок методом послідовної поступки ($f_1 \geq 35824$)

Комбайн	А	В	С	Реальний	За умовою
I	377.6	0	72.4	450	≤ 450
II	0	320	0	320	≤ 320
III	0	300	0	300	≤ 300

Отримано $f_1 = 35824 \text{ м}^3$, $f_2 = 2962.4 \text{ грн}$. Оскільки в задачі лише два критерії, після цього кроку розв'язок вважається остаточним розв'язком методом послідовної поступки.

7 АНАЛІЗ РЕЗУЛЬТАТІВ

Таблиця 7.1 Зведена таблиця розв'язків багатокритеріальної задачі (варіант 6)

Метод	x_{IA}	x_{IB}	x_{IC}	x_{IIA}	x_{IIB}	x_{IIC}	x_{IIIA}	x_{IIIB}	x_{IIIC}	$f_1(x)$	$f_2(x)$
Згортки	0	0	450	0	320	0	0	300	0	39600	3340
Головного критерію	340	0	110	0	320	0	0	300	0	36200	3000
Послідовної поступки	377.6	0	72.4	0	320	0	0	300	0	35824	2962.4

Усі три розв'язки є ефективними за Парето — жоден з них не домінується іншим (кожен покращує один критерій ціною погіршення іншого):

- метод згортки (рівні ваги) дає найбільший обсяг робіт ($f_1 = 39600$) ціною найвищої вартості серед трьох методів ($f_2 = 3340$);
- метод головного критерію дозволяє прямо контролювати максимально допустиму вартість (тут — 3000 грн) і максимізує обсяг робіт у цих межах;
- метод послідовної поступки дає найменшу вартість ($f_2 = 2962.4$) ціною найбільшої поступки за обсягом робіт відносно $f_1^{\max}$.

Остаточний вибір одного з ефективних розв'язків залежить від пріоритетів особи, яка приймає рішення: чи важливіше мінімізувати витрати, чи максимізувати обсяг виконаних робіт. Жоден із трьох методів не претендує на «єдиний правильний» розв'язок — кожен лише пропонує один із раціональних компромісів на множині Парето-оптимальних альтернатив.

8 ВИСНОВКИ

У ході практичної роботи побудовано математичну модель двокритеріальної задачі оптимального завантаження комбайнів для варіанта 6 та розв'язано її трьома методами багатокритеріальної оптимізації, реалізованими програмно мовою Python із застосуванням лінійного програмування (`scipy.optimize.linprog`) замість надбудови Excel «Пошук рішення».

Показано, що метод згортки чутливий до вибору вагових коефіцієнтів: за домівної ваги одного критерію (наприклад, $\alpha_1 = 0.7$) розв'язок вироджується до чистого однокритеріального оптимуму — це пояснюється розділюваністю обмежень задачі за комбайнами. Використання рівнозначних ваг ($\alpha_1 = \alpha_2 = 0.5$) дало змістовний компромісний розв'язок. Методи головного критерію та послідовної поступки дали ще два відмінних Парето-оптимальних розв'язки, кожен з яких по-різному балансує між обсягом робіт і вартістю. Усі три отримані розв'язки є ефективними за Парето, а остаточний вибір між ними лежить поза межами математичної моделі й залежить від пріоритетів особи, яка приймає рішення.

9 ЛІСТИНГ КОДУ

9.1 multi_criteria.py

"""

Практична робота №3 (розділ 2) – Варіант 6

Прийняття рішень в задачах багатокритеріальної оптимізації

Задача: оптимальне завантаження трьох комбайнів (I, II, III), які виконують

три види робіт (A, B, C), за критеріями максимізації сумарного обсягу робіт

(f1) та мінімізації сумарної вартості (f2).

Усі обчислені результати також зберігаються у data.json, який читає безпосередньо Typst-звіт.

"""

```
import json
```

```
import pathlib
```

```
import numpy as np
```

```
from scipy.optimize import linprog
```

```
SCRIPT_DIR = pathlib.Path(__file__).resolve().parent
```

```
def r2(x):
```

```
    if isinstance(x, np.ndarray):
```

```
        return np.round(x, 2).tolist()
```

```
    return round(float(x), 2)
```

```
# -----
```

```
# Вихідні дані (варіант 6, Додаток B)
```

```
# -----
```

```
machines = ["I", "II", "III"]
```

```
jobs = ["A", "B", "C"]
```

```
q = np.array([
    [30, 20, 40],
```

```

        [40, 30, 20],
        [20, 40, 30],
    ], dtype=float) # продуктивність q_ij, м3/год

c = np.array([
    [3, 6, 4],
    [4, 2, 3],
    [5, 3, 2],
], dtype=float) # питома вартість c_ij, грн/год

T = np.array([450.0, 320.0, 300.0]) # ресурс часу, год

q_flat = q.flatten()
c_flat = c.flatten()

A_ub = np.zeros((3, 9))
for i in range(3):
    A_ub[i, i * 3:(i + 1) * 3] = 1
b_ub = T
bounds = [(0, None)] * 9

def solve(coef, extra_A=None, extra_b=None):
    A = A_ub if extra_A is None else np.vstack([A_ub, extra_A])
    b = b_ub if extra_b is None else np.append(b_ub, extra_b)
    res = linprog(coef, A_ub=A, b_ub=b, bounds=bounds, method="highs")
    return res.x

print("=" * 70)
print("Побудова математичної моделі (варіант 6)")
print("=" * 70)
print(f"q (продуктивність) =\n{q}")
print(f"c (питома вартість) =\n{c}")
print(f"T (ресурс часу) = {T}")

# -----
# Знаходження f1_max, f1_min, f2_max, f2_min для нормалізації

```

```

# -----
x_f1max = solve(-q_flat)
f1_max = q_flat @ x_f1max
f1_min = 0.0
f2_at_f1max = c_flat @ x_f1max

x_f2max = solve(-c_flat)
f2_max = c_flat @ x_f2max
f2_min = 0.0

print("\n--- Пошук max/min значень критеріїв для нормалізації ---")
print(f"f1_max = {f1_max} при x = {np.round(x_f1max, 2)}")
print(f"f1_min = {f1_min}")
print(f"f2_max = {f2_max} при x = {np.round(x_f2max, 2)}")
print(f"f2_min = {f2_min}")
print(f"f2 у розв'язку f1_max: {f2_at_f1max}")

# -----
# 2.1 Метод згортки (лінійна адитивна згортка нормованих критеріїв)
# -----
print("\n" + "=" * 70)
print("2.1 Метод згортки")
print("=" * 70)

alpha1, alpha2 = 0.5, 0.5
#  $F(x) = a1 \cdot (f1_{max} - f1(x)) / f1_{max} + a2 \cdot (f2(x) - f2_{min}) / f2_{max} \rightarrow \min$ 
coef_conv = -alpha1 / f1_max * q_flat + alpha2 / f2_max * c_flat
x_conv = solve(coef_conv)
f1_conv = q_flat @ x_conv
f2_conv = c_flat @ x_conv
F_conv = alpha1 * (f1_max - f1_conv) / f1_max + alpha2 * (f2_conv - f2_min) / f2_max

print(f"Вагові коефіцієнти: alpha1={alpha1}, alpha2={alpha2}")
print(f"x = {np.round(x_conv, 2)}")
print(f"f1 = {f1_conv}, f2 = {f2_conv}, F = {F_conv:.6f}")

# для довідки: результат зі значеннями ваг з методички (0.7/0.3) –

```

```

# ілюструє вироджений випадок, коли компроміс збігається з чистим
max(f1)
coef_conv_ref = -0.7 / f1_max * q_flat + 0.3 / f2_max * c_flat
x_conv_ref = solve(coef_conv_ref)
f1_conv_ref = q_flat @ x_conv_ref
f2_conv_ref = c_flat @ x_conv_ref
print(f"\nДовідково, alpha1=0.7, alpha2=0.3: x = {np.round(x_conv_ref,
2)}, "
      f"f1={f1_conv_ref}, f2={f2_conv_ref} "
      f"({'збігається з чистим max(f1)' if np.allclose(x_conv_ref,
x_f1max) else 'відрізняється'})")

# -----
# 2.2 Метод головного критерію
# -----
print("\n" + "=" * 70)
print("2.2 Метод головного критерію")
print("=" * 70)

f2_threshold = 3000.0
x_mc = solve(-q_flat, extra_A=c_flat, extra_b=f2_threshold)
f1_mc = q_flat @ x_mc
f2_mc = c_flat @ x_mc

print(f"Головний критерій: f1 -> max, додаткове обмеження f2(x) <=
{f2_threshold}")
print(f"x = {np.round(x_mc, 2)}")
print(f"f1 = {f1_mc}, f2 = {f2_mc}")

# -----
# 2.3 Метод послідовної поступки
# -----
print("\n" + "=" * 70)
print("2.3 Метод послідовної поступки")
print("=" * 70)

delta_frac = 0.163
delta = round(delta_frac * f1_max)

```

```

f1_floor = f1_max - delta
x_seq = solve(c_flat, extra_A=-q_flat, extra_b=-f1_floor)
f1_seq = q_flat @ x_seq
f2_seq = c_flat @ x_seq

print(f"Крок 1: f1 -> max на всій множині: f1_max = {f1_max} (x =
{np.round(x_f1max, 2)}")
print(f"Поступка Delta = {delta} ({delta_frac*100:.1f}% від f1_max),
f1_floor = {f1_floor}")
print(f"Крок 2: f2 -> min за умови f1(x) >= {f1_floor}")
print(f"x = {np.round(x_seq, 2)}")
print(f"f1 = {f1_seq}, f2 = {f2_seq}")

# -----
# 2.4 Аналіз результатів
# -----
print("\n" + "=" * 70)
print("2.4 Зведена таблиця результатів")
print("=" * 70)

methods = [
    {"name": "Згортки", "x": x_conv, "f1": f1_conv, "f2": f2_conv},
    {"name": "Головного критерію", "x": x_mc, "f1": f1_mc, "f2":
f2_mc},
    {"name": "Послідовної поступки", "x": x_seq, "f1": f1_seq, "f2":
f2_seq},
]
for m in methods:
    print(f"{m['name']:<24} x={np.round(m['x'], 2)} f1={m['f1']:.1f}
f2={m['f2']:.1f}")

# -----
# Збереження результатів у data.json
# -----
data = {
    "setup": {
        "machines": machines,
        "jobs": jobs,

```

```

    "q": q.astype(int).tolist(),
    "c": c.astype(int).tolist(),
    "T": T.astype(int).tolist(),
},
"normalization": {
    "f1_max": r2(f1_max),
    "f1_min": r2(f1_min),
    "x_f1max": r2(x_f1max),
    "f2_max": r2(f2_max),
    "f2_min": r2(f2_min),
    "x_f2max": r2(x_f2max),
    "f2_at_f1max": r2(f2_at_f1max),
},
"convolution": {
    "alpha1": alpha1,
    "alpha2": alpha2,
    "x": r2(x_conv),
    "f1": r2(f1_conv),
    "f2": r2(f2_conv),
    "F": round(float(F_conv), 6),
    "reference_0703": {
        "x": r2(x_conv_ref),
        "f1": r2(f1_conv_ref),
        "f2": r2(f2_conv_ref),
        "matches_f1max": bool(np.allclose(x_conv_ref, x_f1max)),
    },
},
},
"main_criterion": {
    "f2_threshold": f2_threshold,
    "x": r2(x_mc),
    "f1": r2(f1_mc),
    "f2": r2(f2_mc),
},
"sequential": {
    "delta_frac": delta_frac,
    "delta": delta,
    "f1_floor": r2(f1_floor),
    "x": r2(x_seq),

```

```

        "f1": r2(f1_seq),
        "f2": r2(f2_seq),
    },
    "summary": [
        {"method": m["name"], "x": r2(m["x"]), "f1": r2(m["f1"]), "f2":
r2(m["f2"])}
        for m in methods
    ],
}

out_path = SCRIPT_DIR / "data.json"
with open(out_path, "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)
print(f"\nДані збережено у {out_path}")

```